Bước 44 — Production Checklist & Deployment cụ thể

Bước này không thêm nhiều GraphQL feature mới. Mục tiêu là biến project hiện tại thành một hệ thống có thể deploy an toàn, kiểm tra được, rollback được và monitor được.

Ta sẽ đi theo flow:

```
Development
    ↓
RSpec / Security checks
    ↓
Staging
    ↓
Database migration
    ↓
Deploy Backend
    ↓
Sync Trusted Operations
    ↓
Smoke Test
    ↓
Deploy Frontend
    ↓
Monitoring
    ↓
Rollback n□u có l□i
```

1. Kiểm tra cấu trúc production hiện tại

Sau 43 bước, project nên gần như:

```
app/
├── controllers/
│   └── graphql_controller.rb
│
├── graphql/
│   ├── mutations/
│   ├── resolvers/
│   ├── types/
│   ├── sources/
│   └── your_app_schema.rb
│
├── models/
│   ├── user.rb
│   ├── project.rb
│   ├── project_member.rb
│   └── task.rb
│
├── policies/
│   ├── project_policy.rb
│   └── task_policy.rb
│
└── services/
    ├── graphql/
    │   └── request_logger.rb
    ├── projects/
    └── tasks/

config/
├── environments/
│   └── production.rb
├── initializers/
│   └── rack_attack.rb
└── routes.rb

spec/
├── graphql/
├── policies/
├── services/
└── requests/
```

Nếu business logic vẫn nằm rất nhiều trong QueryType hoặc MutationType, nên refactor trước production.

2. Production environment

Kiểm tra: config/environments/production.rb

Rails production phải: RAILS_ENV=production

Có thể kiểm tra:

```
RAILS_ENV=production bin/rails runner 'puts Rails.env'

# K□t qu□:
production
```

3. Environment variables

Production thường cần tối thiểu:

```
DATABASE_URL
SECRET_KEY_BASE
REDIS_URL
RAILS_ENV
```

Tùy authentication/deployment còn có:

```
FRONTEND_URL
SENTRY_DSN
SMTP_*
GRAPHQL_PRO_LICENSE
```

nếu project thực sự sử dụng chúng.

Không viết DATABASE_PASSWORD = "mypassword" trong source code.

Không commit:

```
.env.production
master.key
credentials key
database password
Redis password
API token
```

4. Kiểm tra secrets trước deploy

Có thể kiểm tra:

```
bin/rails credentials:show
```

nhưng tuyệt đối không copy output vào Git, CI logs, Slack, ticket.

Trong production, application chỉ cần nhận secrets qua môi trường hoặc secret manager của platform.

5. Database production

Ví dụ PostgreSQL:

```
Rails
  ↓
DATABASE_URL
  ↓
PostgreSQL
```

Kiểm tra connection:

```
RAILS_ENV=production bin/rails runner 'puts ActiveRecord::Base.connection.database_version'
```

Nếu lỗi ở đây thì chưa cần nghĩ tới GraphQL — database connection chưa hoạt động.

6. Connection pool

Giả sử server:

```
2 Puma workers
×
5 threads
```

thì có thể cần khoảng 10 database connections chỉ riêng web process.

config/database.yml có thể dùng:

```
production:
  url: <%= ENV["DATABASE_URL"] %>
  pool: <%= ENV.fetch("RAILS_MAX_THREADS", 5) %>
```

Nhưng nếu có nhiều workers, Sidekiq/background jobs, console... phải tính tổng connection.

Không làm: Puma concurrency = 50 / DB max connections = 20 rồi mong hệ thống ổn định.

7. Database constraints

Đừng chỉ dựa vào Rails validation.

Ví dụ ProjectMember:

```
validates :user_id,
  uniqueness: {
    scope: :project_id
  }
```

Database cũng cần unique index:

```
add_index :project_members,
          [:project_id, :user_id],
          unique: true
```

Vì production có concurrent requests. Hai request cùng lúc có thể vượt qua model validation trước khi một request commit. Database constraint mới là lớp bảo vệ cuối cùng.

8. Kiểm tra indexes

Project hiện tại nên xem xét các query thực tế như:

```
Task.where(
  project_id: project.id,
  status: "todo"
)
```

Có thể cần:

```
add_index :tasks,
          [:project_id, :status]
```

Assignment:

```
Task.where(
  assignee_id: user.id,
  status: "todo"
)
```

có thể cần:

```
add_index :tasks,
          [:assignee_id, :status]
```

Membership:

```
add_index :project_members,
          [:project_id, :user_id],
          unique: true
```

Nhưng đừng index mọi column. Dùng relation.explain để kiểm tra query plan.

9. Migration production phải an toàn

Trước deploy:

```
bin/rails db:migrate
```

Nhưng production migration không nên chỉ nghĩ: migration chạy được local → production cũng OK.

Ví dụ nguy hiểm trên bảng lớn:

```
add_column :tasks,
           :status,
           :string,
           null: false,
           default: "todo"
```

Tùy database/version/table size, thay đổi lớn có thể gây lock hoặc ảnh hưởng deploy.

Nên nghĩ theo nhiều bước:

```
add nullable column
↓
deploy
↓
backfill
↓
validate data
↓
add constraint
```

10. Không rename/drop column quá sớm

Bước 43 đã học Schema Evolution.

Ví dụ muốn projects.name → projects.title.

Đừng deploy một lần:

```
rename_column :projects,
              :name,
              :title
```

trong khi code cũ vẫn đang chạy.

An toàn hơn:

```
Deploy A
→ thêm title

Deploy B
→ dual read/write hoặc migrate

Deploy C
→ frontend dùng title

Deploy D
```

```
→ remove name khỏi GraphQL

Deploy E
→ remove DB column cũ
```

11. Authentication production

GraphqlController:

```
context = {
  current_user: current_user
}
```

Nhưng điều này không tự bảo vệ GraphQL.

Resolver/mutation vẫn phải:

```
authenticate_user!
```

Ví dụ:

```
def resolve
  user = authenticate_user!

  # ...
end
```

12. Authorization production

Mutation:

```
def resolve(id:, attributes:)
  authenticate_user!

  project =
    Project.find(id)

  authorize!(
    project,
    :update?
  )

  # ...
end
```

Collection:

```
ProjectPolicy::Scope
  .new(current_user, Project)
  .resolve
```

Điểm cần review: Single record → Policy; Collection → Policy::Scope.

Không chỉ test admin. Phải test: admin, owner, manager, developer, viewer, stranger.

13. Chạy Permission Matrix

Trước production:

```
bundle exec rspec spec/policies
```

và permission matrix specs.

Ví dụ:

```
                admin owner manager dev viewer stranger

View Project      ✓     ✓     ✓      ✓    ✓      ✗
Update Project    ✓     ✓     ✓      ✗    ✗      ✗
Delete Project    ✓     ✓     ✗      ✗    ✗      ✗
Manage Members    ✓     ✓     ✓      ✗    ✗      ✗
Create Task       ✓     ✓     ✓      ✓    ✗      ✗
Delete Task       ✓     ✓     ✓      ✗    ✗      ✗
```

Một permission regression production nguy hiểm hơn rất nhiều so với UI bug.

14. Pagination production

Không expose collection vô hạn mà không có giới hạn phù hợp.

Client nên:

```
query GetTasks(
  $first: Int!
  $after: String
) {
  tasks(
    first: $first
    after: $after
  ) {
    nodes {
      id
      title
```

```
    }

    pageInfo {
      hasNextPage
      endCursor
    }
  }
}
```

Server phải có max page size phù hợp.

15. N+1 production check

Query:

```
projects {
  nodes {
    owner {
      email
    }

    tasks {
      nodes {
        assignee {
          email
        }
      }
    }
  }
}
```

Review: Project.owner, Project.tasks, Task.assignee, Project.members để chắc chắn Dataloader/batching đang hoạt động.

Một query trả đúng data nhưng tạo 500 SQL queries vẫn là bug production.

16. Query Depth / Complexity

Schema production phải giữ các giới hạn đã học ở Bước 39.

Concept:

```
GraphQL request
      ↓
depth validation
      ↓
complexity validation
      ↓
execution
```

Không remove limit chỉ vì frontend hiện tại “không query sâu”. Client vẫn có thể gửi request bất thường.

17. Rate limiting

Bước 40: config/initializers/rack_attack.rb

Concept:

```
class Rack::Attack
  throttle(
    "graphql/ip",
    limit: 100,
    period: 1.minute
  ) do |request|
    request.ip if
      request.post? &&
      request.path == "/graphql"
  end
end
```

100/minute chỉ là ví dụ khởi đầu. Production phải điều chỉnh theo traffic.

18. Shared cache cho Rate Limit

Nếu production có Rails A, Rails B, Rails C thì không nên mỗi instance tự giữ counter riêng.

Cần shared cache, ví dụ Redis-backed cache phù hợp với hạ tầng.

19. Trusted proxy

Nếu Browser → Cloudflare → Nginx → Rails, phải kiểm tra request.ip thực sự là client IP.

Nếu không, mọi request có thể cùng proxy IP và Rack::Attack có thể throttle nhầm toàn bộ user.

20. CORS

Nếu frontend https://app.example.com và backend https://api.example.com, CORS production chỉ nên cho origin cần thiết.

Concept: origins ENV.fetch("FRONTEND_URL").

Không mặc định origins "*", đặc biệt khi authentication dùng credentials/cookies.

21. CSRF và Devise

Bạn đang dùng Devise nên cần xác định rõ Devise session cookie hay Bearer/JWT/token.

Nếu dùng session cookie: Browser → Cookie → Rails, thì CSRF protection rất quan trọng.

Đừng copy tutorial API rồi vô hiệu hóa skip_forgery_protection mà chưa hiểu authentication architecture.

22. GraphiQL production

Development: /graphiql → OK.

Production: /graphiql → thường không public.

Routes có thể conditional:

```
if Rails.env.development?
  mount GraphiQL::Rails::Engine,
        at: "/graphiql",
        graphql_path: "/graphql"
end
```

23. OperationStore / Trusted Documents

Nếu Bước 42 được triển khai: Production client → operationId → OperationStore → trusted document.

Deploy order phải là Backend → Sync operations → Frontend.

Không deploy Frontend trước operationId mới khi server chưa biết.

24. Logging

Bước 41 RequestLogger nên cho output kiểu:

```
{
  "event": "graphql.execute",
  "operation": "GetTasks",
  "request_id": "abc123",
  "user_id": 15,
  "duration_ms": 84.32,
  "errors": 0,
  "error_codes": [],
  "slow": false
}
```

Nếu Bước 42, operation_id cũng có thể được thêm.

25. Không log sensitive data

Review toàn project:

```
grep -R "Rails.logger" app/
```

Tìm những thứ như Rails.logger.info(params.inspect), Authorization header hoặc variables.

Production không nên log password, access token, refresh token, Authorization, sensitive variables.

26. Monitoring

Sau deploy cần biết: requests/minute, errors/minute, p95 latency, p99 latency, slow operations, rate limited requests, DB latency, unexpected exceptions.

Ví dụ: GetMe p95 = 40 ms; GetTasks p95 = 180 ms; GetDashboard p95 = 1.8 s → GetDashboard cần điều tra.

27. Health endpoint

Health check không nên là query dashboard vì quá nhiều dependency.

Dùng health endpoint riêng. Rails version hiện tại có thể đã có /up; nếu project của bạn có route đó, kiểm tra:

```
curl https://api.example.com/up

# Kỳ vọng:
HTTP 200
```

28. RSpec trước deploy

Chạy toàn bộ:

```
bundle exec rspec
```

Không chỉ bundle exec rspec spec/graphql vì GraphQL phụ thuộc Models, Policies, Services, Database behavior.

29. Chạy riêng security tests

Có thể tổ chức:

```
spec/graphql/security/
├── authorization_spec.rb
├── query_complexity_spec.rb
├── query_depth_spec.rb
└── rate_limit_spec.rb
```

Sau đó CI chạy:

```
bundle exec rspec   spec/graphql/security
```

Nếu security specs fail: STOP DEPLOY.

30. CI pipeline

Pipeline cơ bản: git push → CI → bundle install → database setup → RSpec → security tests → build → deploy.

Ví dụ conceptual GitHub Actions:

```
steps:
  - uses: actions/checkout@v4

  - name: Install dependencies
    run: bundle install

  - name: Prepare database
    run: bin/rails db:test:prepare

  - name: Run specs
    run: bundle exec rspec
```

Deployment cụ thể phụ thuộc platform nên chưa nên thêm lệnh Fly/Render/Heroku/AWS khi chưa biết bạn đang dùng gì.

31. Staging

Trước production: development → test → staging → production.

Staging nên gần production nhất có thể: PostgreSQL, Redis, Rack::Attack, GraphQL complexity, GraphQL depth, CORS, authentication.

Không nên staging SQLite + không Redis + không rate limit trong khi production dùng hoàn toàn khác.

32. Smoke test sau deploy

Sau backend deploy, chạy query đơn giản:

```
query GetMe {
  me {
    user {
      id
      email
      role
    }
  }
}
```

Sau đó:

```
query GetProjects(
  $first: Int!
) {
  projects(first: $first) {
    nodes {
      id
      name
    }
  }
}
```

Variables:

```
{
  "first": 5
}
```

Kiểm tra: authentication ✓, GraphQL ✓, database ✓, authorization ✓, pagination ✓.

33. Smoke test Mutation

Không test destructive mutation trên dữ liệu thật quan trọng.

Có thể tạo test project/user dành riêng cho smoke tests.

Ví dụ:

```
mutation CreateTask(
```

```
  $input: CreateTaskInput!
) {
  createTask(input: $input) {
    task {
      id
      title
    }

    errors
  }
}
```

Sau test: create → verify → cleanup.

34. Monitor ngay sau deploy

Khoảng thời gian ngay sau deploy cần nhìn: error rate, latency, DB errors, FORBIDDEN spike, VALIDATION_ERROR spike, 429 spike, 500 spike.

Ví dụ trước deploy VALIDATION_ERROR 5/hour, sau deploy 500/hour → khả năng frontend/backend contract vừa bị phá.

35. Rollback plan

Trước deploy phải trả lời được: Code rollback thế nào? Frontend rollback thế nào? DB migration có backward compatible không? OperationStore operations cũ còn chạy không?

Không đợi production lỗi rồi mới nghĩ rollback.

36. Migration rollback không phải lúc nào cũng an toàn

Ví dụ remove_column :tasks, :legacy_status. Sau đó bin/rails db:rollback không thể tự khôi phục dữ liệu đã bị mất.

Do đó destructive migrations phải đặc biệt cẩn thận.

Ưu tiên:

```
deprecate
↓
stop reading
↓
stop writing
↓
wait
↓
remove
```

37. Deployment flow cuối cùng

Đây là flow mình khuyên dùng cho project hiện tại:

```
1. Developer
   ↓
feature branch

2. CI
   ↓
RSpec
   ↓
Permission Matrix
   ↓
Security Specs

3. Merge

4. Deploy Staging
   ↓
db:migrate
   ↓
smoke tests

5. Review staging

6. Deploy Production Backend
   ↓
safe migrations

7. Sync Trusted Operations
   ↓
n□u dùng Bước 42

8. Health Check

9. GraphQL Smoke Tests

10. Deploy Frontend
    ↓
n□u frontend phụ thuộc schema mới

11. Monitor
```

```
    ↓
errors
latency
DB
429

12. Rollback n□u c□n
```

38. Checklist trước khi bấm Deploy

```
CODE
[ ] bundle exec rspec xanh
[ ] Không debug code
[ ] Không secrets trong Git

DATABASE
[ ] Migration reviewed
[ ] Indexes reviewed
[ ] Constraints reviewed
[ ] DB pool phù hợp
[ ] Backup/rollback strategy

GRAPHQL
[ ] Authentication
[ ] ProjectPolicy
[ ] TaskPolicy
[ ] Policy::Scope
[ ] Permission Matrix
[ ] Pagination
[ ] Max page size
[ ] Dataloader
[ ] Complexity
[ ] Depth limit

SECURITY
[ ] Rack::Attack
[ ] CORS
[ ] CSRF strategy
[ ] GraphiQL không public tùy tiện
[ ] Dashboard admin-only
[ ] Không log token/password

OBSERVABILITY
[ ] operation name
[ ] request_id
[ ] user_id
[ ] duration_ms
[ ] errors
[ ] error_codes
[ ] slow query detection

DEPLOYMENT
[ ] Staging passed
[ ] Backend deploy order
[ ] OperationStore sync n□u dùng
[ ] Health check
[ ] Smoke test
[ ] Frontend deploy
[ ] Monitoring
[ ] Rollback plan
```

39. Definition of Done cho Bước 44

Bước 44 không phải hoàn thành khi app deploy được.

Nó hoàn thành khi:

```
App deploy được
        +
Tests xanh
        +
Permission được ki□m chứng
        +
Migration an toàn
        +
Security limits hoạt động
        +
Logs/metrics hoạt động
        +
Có health check
        +
Có smoke tests
        +
Có rollback plan
```

Zusammenfassung — Code

Đây là phần code/checklist ngắn để bạn lưu lại.

Schema

```
class YourAppSchema < GraphQL::Schema
```

```
  query Types::QueryType
  mutation Types::MutationType

  use GraphQL::Dataloader

  # Depth / Complexity:
  # c□u hình theo version graphql-ruby
  # và s□ liệu thực t□ c□a project.

  # N□u dùng GraphQL Pro:
  # use GraphQL::Pro::OperationStore
end
```

Controller context

```
context = {
  current_user: current_user
}

result =
  YourAppSchema.execute(
    query,
    variables: variables,
    context: context,
    operation_name: operation_name
  )
```

Authorization

```
user =
  authenticate_user!

project =
  Project.find(id)

authorize!(
  project,
  :update?
)
```

Collection scope

```
ProjectPolicy::Scope
  .new(
    current_user,
    Project
  )
  .resolve
```

Database constraint

```
add_index :project_members,
          [:project_id, :user_id],
          unique: true
```

Useful task indexes

```
add_index :tasks,
          [:project_id, :status]

add_index :tasks,
          [:assignee_id, :status]
```

Rate limit

```
class Rack::Attack
  throttle(
    "graphql/ip",
    limit: 100,
    period: 1.minute
  ) do |request|
    request.ip if
      request.post? &&
      request.path == "/graphql"
  end
end
```

Logging

```
{
  event: "graphql.execute",
  operation: operation_name,
  request_id: request.request_id,
  user_id: current_user&.id,
  duration_ms: duration_ms.round(2),
  errors: error_count,
  error_codes: error_codes,
  slow: slow?
}
```

Test

```
bundle exec rspec
```

Production migration

```
RAILS_ENV=production bin/rails db:migrate
```

Health check

```
curl https://api.example.com/up
```

Deploy

```
RSpec
  ↓
Staging
  ↓
Migration
  ↓
Backend
  ↓
OperationStore Sync
  ↓
Health Check
  ↓
GraphQL Smoke Test
  ↓
Frontend
  ↓
Monitoring
```

Câu quan trọng nhất của Bước 44:

```
Production-ready
≠
"Code chạy được"

Production-ready
=
Code
+ Tests
+ Security
+ Database safety
+ Observability
+ Deployment strategy
+ Rollback
```

Sau Bước 44, Bước 45 mới là lúc gom toàn bộ Project Management API thành kiến trúc cuối cùng, thay vì tiếp tục thêm security/deployment feature rời rạc.